

Code Explanation

Test DLT Pipeline Code Explanation

This is a test suite written in Python using the unittest framework to test a data pipeline built using Databricks Delta Lake (DLT). The pipeline is responsible for processing customer and order data.

Overview of the Code

The code defines a test class `TestDLTPipeline` that contains several test methods to validate the functionality of the DLT pipeline. The pipeline is composed of several functions that read data from CSV files, perform data processing, and produce output DataFrames. The test class mocks the DLT module and tests the pipeline functions using sample data.

Step 1: Setting Up the Test Environment

The `setUpClass` method is a class-level setup method that is called once before running all test methods. It creates a local Spark session, sets up temporary directories for test data, and creates sample customer and order data.

```
cls.spark = SparkSession.builder
    .appName("DLTPipelineTest")
    .master("local[*]")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .getOrCreate()
```

Step 2: Creating Test Data

The `\_create\_test\_data` method creates sample customer and order data as Pandas DataFrames and saves them to CSV files in the temporary directories.

```
customer_data = [
    {"CustId": "C001", "Name": "John Doe", "EmailId": "john@example.com", "Region": "North"},
    # ...
]

customer_df = pd.DataFrame(customer_data)
customer_df.to_csv(os.path.join(cls.customer_data_path, "customer.csv"), index=False)
```

Step 3: Mocking the DLT Module

The `setUpClass` method mocks the DLT module by creating a mock object with the required functions (`table`, `read`, and `expect\_all\_or\_drop`). This allows the test class to test the pipeline functions without actually using the DLT module.

```
sys.modules['dlt'] = type('obj', (object,), {
    'table': lambda **kwargs: lambda f: f,
    'read': lambda table name: cls.mock_dlt_read(table name),
```

```
'expect_all_or_drop': lambda expectations: lambda f: f
})
```

Step 4: Testing the Customer Table Function

The ``test_customer_table`` method tests the ``customer`` function by overriding the data path, running the function, and verifying the output DataFrame.

```
result = self.customer_func()
self.assertEqual(result.count(), 5) # After removing duplicates and
nulls
```

Step 5: Testing the Order Table Function

The ``test_order_table`` method tests the ``order`` function by overriding the data path, running the function, and verifying the output DataFrame.

```
result = self.order_func()
self.assertEqual(result.count(), 5) # After removing duplicates and
nulls
```

Step 6: Testing the Customer Aggregate Spend Function

The ``test_customeragggregatespend`` method tests the ``customeragggregatespend`` function by running the function and verifying the output DataFrame.

```
result = self.customeragggregatespend_func()
john_spend = result.filter("Name = 'John Doe'").collect()
```

Step 7: Cleaning Up

The ``tearDownClass`` method is a class-level teardown method that is called once after running all test methods. It stops the Spark session and removes the temporary directories.

```
cls.spark.stop()
shutil.rmtree(cls.test_dir)
```